

第1章. 认识项目与 Agent / RAG 基础

本章我们从一个实际问题出发：维修人员输入“东风天龙仪表针脚图”，系统是怎样理解问题、查找资料，并在结果太多时继续询问的？

先把这条流程看懂，再学习里面的模型、工具和检索组件。这样，后面阅读代码时，每个名词都有对应的业务位置。

学习目标

1. 区分大语言模型、Agent、RAG 和工作流。
2. 看懂项目中的模型调用和消息格式。
3. 理解资料从 CSV 变成 Document 的过程。
4. 找到 Web 主流程与独立 Agent 的入口。
5. 按统一命名和注释方式运行本章的小实验。

本章范围： 项目入门与概念验证。混合检索算法、查询优化、完整图编排和质量评估留到后续章节，不要求本章一次掌握。

交付方式： 后续章节统一使用 PDF。代码直接收录在文档中，不另附学习脚本。需要运行实验时，请在项目根目录自行新建对应练习文件，复制本节完整代码后执行；示例文件名是建议命名，不表示这些文件已经存在。

阅读方式： 每节按照“问题 → 名词解析 → 应用场景 → 项目对应 → 代码示例 → 结果解释”展开。代码保留课程常用的分步注释；示例复用项目已有模块，不替换业务实现。

代码基准： 2026-10-03 本地项目源码。模型名称和配置以 `app/core/config.py` 及你当前环境为准；接口用法已结合官方文档核对。

本章目录

- 1. 认识我们的项目 3
- 2. 模型 (Model) : 谁负责理解, 谁负责检索 5
- 3. 消息 (Messages) 与提示词 (Prompt) 7
- 4. RAG: 让系统找到真实资料 10
- 5. Agent: 让模型使用工具 14
- 6. LangGraph: 把检索过程组织成工作流 18
- 7. 命名规范与代码阅读顺序 21
- 8. 动手练习与本章回顾 23
- 附录. 本章来源与阅读定位 27

1. 认识我们的项目

1.1 需求分析

假设你是一名车辆维修人员，现在需要查找某个车型的仪表针脚资料。

你可能知道完整型号，也可能只记得品牌、车型简称和部件。你希望直接说一句话，系统就能帮助你定位资料，而不是逐层翻目录。

这个项目要解决的就是这类**资料查询与导航**问题：

- **自然语言提问**：输入品牌、车型、部件或 ECU 型号。
- **资料检索**：从已有资料库中找到相关记录。
- **多轮澄清**：结果较多时，让用户通过分类选项继续缩小范围。
- **结果展示**：返回真实资料的 ID、标题和层级路径。

例如，用户可以这样提问：

代码块 text

- 1 东风天龙仪表针脚图
- 2 徐工 XE135GII 仪表显示器针脚定义
- 3 康明斯 2880 电路图

这些是查询示例，不代表每个问题在当前数据、配置下都一定得到固定数量的结果。

1.2 项目的能力边界

我们先把一个容易混淆的地方说清楚：**找到电路图资料，不等于读懂电路图并诊断故障。**

当前的数据入口是 `data/circuit-data.csv`。程序主要把“资料标题”和“层级路径”作为检索文本，最终返回资料信息供用户定位。

本章讲的是这条已存在的链路：

代码块 text

- 1 用户描述要找的资料
- 2 ↓
- 3 系统理解并检索
- 4 ↓
- 5 返回真实资料记录

当前主链路不包含电路图 OCR、图像连线识别或依据完整图纸生成维修诊断。后续如果加入这些能力，需要新的数据处理和验证流程。

1.3 实现思路

如果只让大模型回答“有没有某张电路图”，模型未必知道你的资料库里究竟有什么，也无法保证返回真实的资料 ID。

因此，我们需要把几种能力组合起来：

- **大语言模型**：理解用户描述，识别检索意图，辅助改写查询。
- **检索程序**：从资料库中实际查找候选记录。
- **workflow**：根据候选数量，决定直接返回还是继续澄清。
- **Web 接口**：接收问题、展示选项和资料卡片。

这就是本章要认识的大模型应用：模型处理自然语言，程序执行检索与业务规则，两者协作完成任务。

概念	先问哪个问题	本项目对应	不应误解为
LLM	用户在说什么？	理解、改写、对话	自动掌握本地资料库
RAG	外部依据从哪里来？	真实资料检索	更换或重新训练模型
Agent	下一步调用什么能力？	独立模型与工具循环	模型亲自执行 Python
Workflow	业务步骤怎样可靠衔接？	Web 自定义 LangGraph	每个节点都是模型调用

1.4 先看整体流程

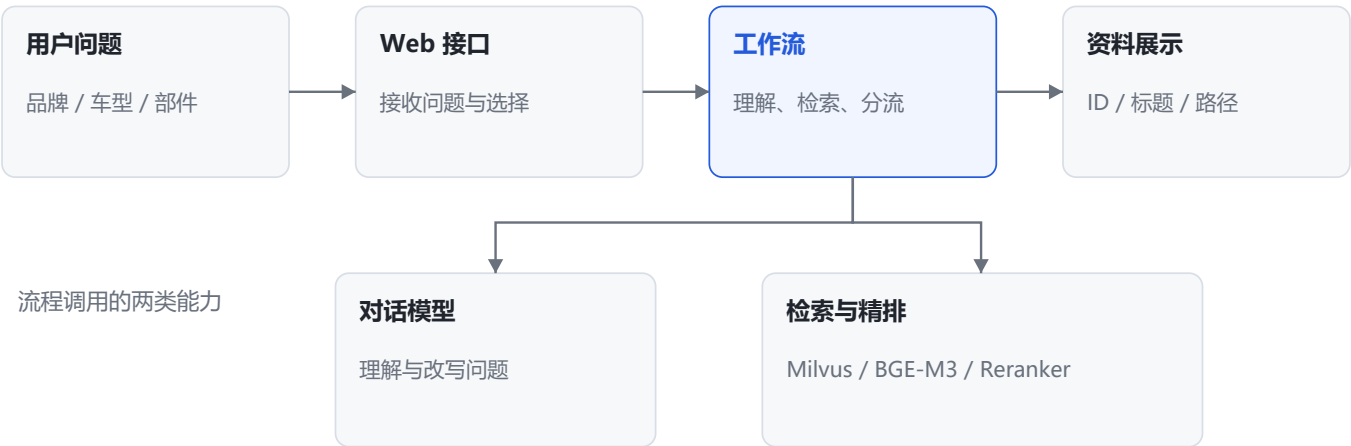


图1-1 项目全貌：自然语言理解、真实资料检索与业务分支协作

这里的 5 条是项目默认展示上限，可通过 GRAPH_MAX_RESULTS 配置。它是展示规则，不是检索准确率指标。

注意

主流程会保留比最终展示数量更多的候选，以便生成分类选项。不要把“召回候选数”和“最终展示数”当作同一个参数。

1.5 本节回顾

现在，你应该能够用自己的话说明：项目服务谁、输入是什么、最终输出什么，以及为什么不能只依靠模型记忆查找内部资料。

2. 模型（Model）：谁负责理解，谁负责检索

2.1 什么是 LLM

LLM, Large Language Model, 大语言模型，是一类能够理解和生成自然语言的模型。

在应用开发中，我们通常调用已经训练好的模型服务，不需要先训练一个自己的模型。项目通过 DeepSeek API 完成对话与查询理解。

但请记住：模型生成的内容不天然等于资料库事实。它可能误解型号，也可能生成没有依据的内容，所以实际资料必须来自检索结果。

应用场景： 理解“我想找天龙的仪表针脚资料”这类口语表达，判断是不是资料查询，提取品牌、车型、部件等信息。

项目对应： app/core/model.py、app/rag/intent_parser.py。

2.2 为什么项目里不止一个模型

启动项目后，你可能会看到本地模型加载，又在代码里看到 DeepSeek。它们负责的事情不同。

名词	本项目默认实现	输入与输出	主要职责
对话模型（Chat Model）	DeepSeek API	消息 → 回复或结构化信息	理解问题、改写查询、普通对话
向量模型（Embedding Model）	BGE-M3, 本地加载	文本 → 数值向量	支持语义检索
精排模型（Reranker）	Qwen3-Reranker-0.6B, 本地加载	问题与候选文本 → 相关性得分	对已召回资料重新排序

Embedding, 向量化，就是把文本编码为一组数值，让程序能够计算文本之间的相似程度。它的输出不是自然语言回答。

Reranker, 重排序模型，会同时考虑问题和候选文本，判断候选是否相关。它处理的是召回之后的候选，不是独自遍历整个资料库。

例如，同一个问题的处理顺序可以理解为：



图1-2 三类模型的输入、职责和输出各不相同

图1-2 展示输入与输出的区别，不是三类模型必须串行执行的流程。具体调用顺序以实际入口和检索节点为准。

2.3 如何初始化对话模型

课程使用 `init_chat_model()` 统一初始化模型。项目也采用相同写法，只是把配置、密钥检查和缓存封装进了 `get_chat_model()`。

源码阅读： 打开 `app/core/model.py`，重点看以下调用。下面是阅读片段，`settings` 来自同文件中的 `get_settings()`：

代码块 python

```
1 # 1. 通过统一接口初始化模型
2 model = init_chat_model(
3     model=settings.deepseek_model,
4     model_provider="deepseek",
5     api_key=settings.deepseek_api_key.get_secret_value(),
6     base_url=settings.deepseek_api_base,
7     temperature=0.1,
8     timeout=30,
9     max_retries=2,
10 )
```

参数可以这样理解：

- `model`：调用哪个模型；项目默认是 `deepseek-chat`。
- `model_provider`：选择对应模型集成；这里是 `deepseek`。
- `api_key`：访问服务需要的凭证，由配置读取。
- `base_url`：服务接口的基础地址。
- `temperature`：控制采样随机性的参数；较低值不意味着结果必然正确或完全一致。
- `timeout`：单次请求的超时配置，单位为秒。
- `max_retries`：客户端的重试配置；重试不能代替业务降级处理。

这里的 `get_secret_value()` 是读取 `SecretStr` 保存的值。学习时不需要打印这个值。

2.4 API、SDK 与框架是什么关系

API, Application Programming Interface, 应用程序编程接口，是程序访问模型服务的接口。

SDK, Software Development Kit, 软件开发工具包，把发送请求、解析响应等工作封装成可以调用的代码。

LangChain 在此基础上提供统一的模型、消息和工具接口。它没有把请求变成本地推理，真正的模型计算仍发生在对应服务中。

代码块 text

```
1  你的 Python 代码
2      ↓
3  LangChain 模型接口
4      ↓
5  模型集成与客户端
6      ↓
7  DeepSeek HTTP API
```

2.5 本节回顾

试着回答：为什么本地加载 BGE-M3 时会出现模型权重进度，而调用 DeepSeek 时通常不会？为什么 Reranker 不能替代资料库和检索程序？

3. 消息（Messages）与提示词（Prompt）

3.1 什么是消息

模型调用并不是只发送一句字符串。我们可以给模型发送一组有角色的消息，让它区分任务要求、用户问题、历史回复和工具结果。

Message, 消息，通常包含 `role` 和 `content` 两项：

- `role`：谁提供这条消息。
- `content`：消息的内容。

应用场景： 给车辆助手设定职责，再传入用户问题；或者在下一次模型调用时补充前面的对话背景。

角色 (role)	含义	本项目中的例子
system	系统给模型的角色与任务约束	你是车辆资料导航助手，不编造资料
user	用户提出的问题或要求	东风天龙仪表针脚图
assistant	模型生成的消息	文本回复，或带有工具调用信息的消息
tool	程序执行工具后的结果	资料 ID、标题和层级路径

注意 assistant 的拼写。它不是 assisant。

3.2 什么是提示词

Prompt, 提示词, 是发送给模型的任务说明、上下文和输入。系统提示词通常用来约束角色、任务和回答方式。

例如, 车辆资料导航助手可以使用这样的系统消息:

代码块 python

```
1 system_prompt = """你是车辆资料导航助手。  
2 请帮助用户查找真实存在的车辆资料。  
3 资料的 ID、标题和层级路径必须来自工具结果。  
4 不知道或没有找到时, 明确说明, 不编造资料。  
5 """
```

它告诉模型应该怎样回答。但**提示词不是数据库, 也不是强制执行业务规则**。例如“最多展示 5 条”这类硬限制, 项目还会在程序中检查和处理。

3.3 代码示例: 第一次模型调用

我们先只调用模型, 不启动完整检索。这可以把“模型接口是否能正常工作”和“资料检索是否正常”分开理解。

运行条件: 项目已安装依赖, 根目录 .env 已配置有效的 DEEPSEEK_API_KEY, 并能访问模型服务。这个实验会调用远程 API, 可能产生调用费用; 不需要 Milvus。

练习文件名 (自行新建): ch01_model_call.py。

代码块 python

```
1 from app.core.model import get_chat_model
2
3
4 def main() -> None:
5     # 1. 初始化项目已经配置好的对话模型
6     model = get_chat_model()
7
8     # 2. 准备系统消息和用户消息
9     messages = [
10         {
11             "role": "system",
12             "content": "你是车辆资料导航助手。只解释能力，不编造资料。",
13         },
14         {"role": "user", "content": "请用两句话介绍你能帮助我做什么。"},
15     ]
16
17     # 3. 调用模型，等待完整回复
18     response = model.invoke(messages)
19
20     # 4. 查看消息类型和回复正文
21     print(type(response).__name__)
22     print(response.content)
23
24
25 if __name__ == "__main__":
26     main()
```

在项目根目录打开 PowerShell，执行：

代码块 powershell

```
1 Set-Location -LiteralPath 'D:\Study\chatbot-circuit-diagram'
2 .\.venv\Scripts\python.exe ch01_model_call.py
```

结果解释： 首先应看到消息对象的类型，通常为 `AIMessage`；接着是模型回复正文。具体文字由模型生成，不要求与某个示例答案逐字一致。

这个实验没有检索资料，因此回复只能用于观察模型的角色和表达，不能作为某张资料存在的证明。

3.4 invoke 与 ainvoke 有什么区别

项目里有些地方用 `invoke()`，有些用 `ainvoke()`。区别在调用方式：

- `invoke()`：同步调用，当前执行流程等待结果。
- `ainvoke()`：异步调用，通常配合 `await` 使用。
- `stream()`：迭代接收流式结果，和是否异步是不同的维度。

因此，异步不等于流式。`await model.ainvoke(...)` 仍可以等待完整结果，不代表已经逐个字发送给前端。

本章采用同步实验，方便观察输入输出；项目中的 Web 工作流使用异步入口。

3.5 会话记忆为什么需要程序管理

DeepSeek 的 Chat Completions API 是无状态接口。一次请求不会自动知道另一次请求里说过什么。要让模型根据前文回答，需要向本次请求提供相关历史。

例如：

代码块 python

```
1 messages = [  
2     {"role": "system", "content": "你是车辆资料导航助手。"},  
3     {"role": "user", "content": "我在找东风天龙的资料。"},  
4     {"role": "assistant", "content": "请补充部件或资料类型。"},  
5     {"role": "user", "content": "仪表针脚定义。"},  
6 ]
```

这里通过消息列表补充背景，但这段代码只是消息格式示例，不是当前 Web 的完整记忆实现。无状态含义可参考 [DeepSeek 多轮对话文档](#)。

项目对应：当前 Web 的多轮澄清主要保存候选集、选项和选择历史等工作流状态。每次新问题还会重置候选状态，不应把它描述为已经具备无限聊天历史或长期用户偏好记忆。

3.6 本节回顾

现在，你应该能说明 messages 的结构、系统提示词的用途、同步与异步的区别，以及为什么模型的多轮理解需要本次输入包含背景信息。

4. RAG：让系统找到真实资料

4.1 为什么需要外部资料

假设用户问：“你们资料库里有没有东风天龙仪表针脚图？”

模型可能知道东风天龙是什么，但它没有天然获得你本地 CSV 的内容，也不知道哪条记录在当前资料库中存在。

因此，在涉及资料事实时，我们需要先查询真实数据，再根据结果处理用户请求。

RAG, Retrieval-Augmented Generation, 检索增强生成，通常指先检索外部知识，再把检索内容提供给模型，帮助它生成有依据的回答。

这是一种应用架构思路，不是一种对话模型，也不等于某个框架。

应用场景：企业知识问答、内部资料查询、专业文档助手等，需要依据外部资料回答的问题。

4.2 通用 RAG 与本项目的区别

通用的文本问答 RAG 可以简化为：

代码块 text

```
1 用户问题 → 检索知识片段 → 组织上下文 → 模型生成答案
```

当前项目借用了 RAG 的外部知识检索能力，但 Web 主链路更准确地说是**资料导航 workflow**：

代码块 text

1 用户问题 → 理解与改写 → 检索资料记录 → 澄清或展示资料

为什么要区分？因为 `app/graph/nodes/answer.py` 的最终资料回复由程序组织，并不是把完整电路图正文交给模型生成专业问答。

这是阅读项目代码时的重要判断。仓库中保留的独立 Agent 可以根据工具结果生成回复，但它和当前 Web 展示链路是不同入口。

4.3 Document：检索系统里的资料单位

Document，文档对象，是 LangChain 中表示一条检索文本及其附加信息的结构。最常用的两个字段是：

- `page_content`：参与检索或作为上下文的文本。
- `metadata`：文本对应的附加属性，例如 ID、标题、来源和分类路径。

请注意，Document 不一定是一整个 PDF。它可以是一段正文，也可以是 CSV 中的一条记录。

项目对应： `app/rag/loader.py`。

本项目把每条有效 CSV 记录转换成一个 Document：

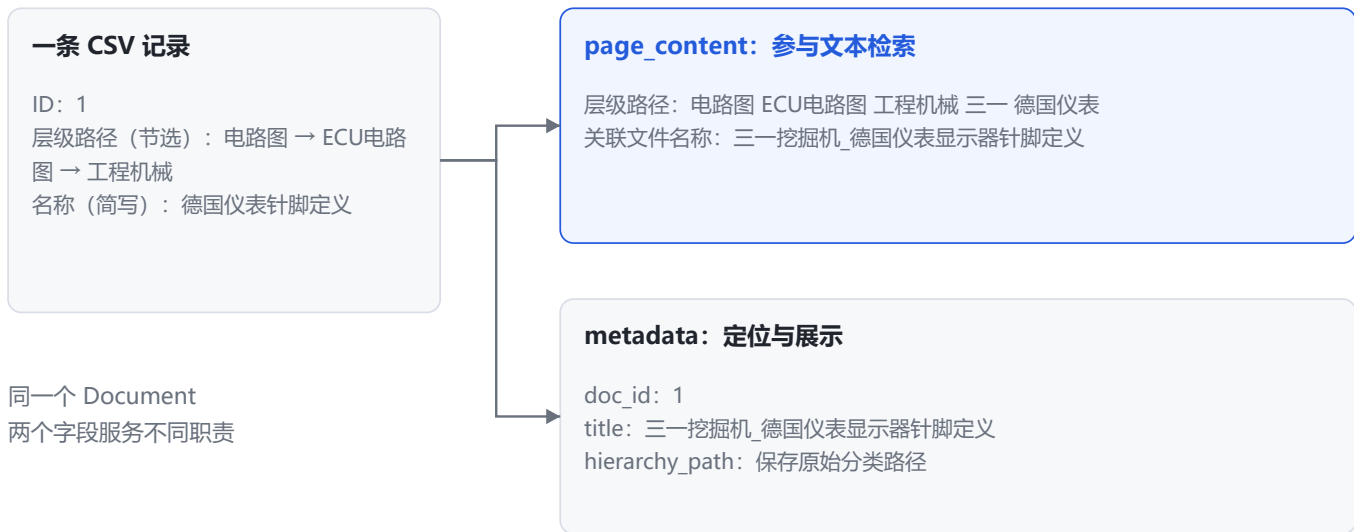


图1-3 同一条资料同时拥有检索文本和结构化元数据

课程中的长文档可能需要 Text Splitter 切分器；这里每条记录已经是独立、较短的资料单位，不必为了使用课程组件再强行切段。

4.4 代码示例：看看真实 Document

我们先看数据，不调用模型，也不写入 Milvus。

运行条件： 项目依赖已安装，`data/circuit-data.csv` 存在。

练习文件名（自行新建）： `ch01_document_preview.py`。

代码块 python

```
1 from app.rag.loader import load_circuit_documents
2
3
4 def main() -> None:
5     # 1. 读取项目的 CSV，不写入数据库
6     result = load_circuit_documents()
7     print(f"有效资料: {result.valid_rows}")
8     print(f"无效记录: {result.invalid_rows}")
9
10    if not result.documents:
11        raise RuntimeError("CSV 没有可用的资料记录")
12
13    # 2. 查看一条 Document 的检索文本
14    doc = result.documents[0]
15    print("\npage_content: ")
16    print(doc.page_content)
17
18    # 3. 查看同一条 Document 的结构化元数据
19    print("\nmetadata: ")
20    for key in ("doc_id", "title", "hierarchy_path"):
21        print(f"{key} = {doc.metadata[key]}")
22
23
24 if __name__ == "__main__":
25     main()
```

运行命令：

代码块 powershell

```
1 .\.venv\Scripts\python.exe ch01_document_preview.py
```

结果解释： 你会看到有效资料数、无效记录数，然后看到一条资料的检索文本与元数据。资料数量和首条内容以你的实际 CSV 为准。

请特别观察：page_content 展示的是资料标题和层级路径，不是电路图图片中的线路或针脚表正文。

项目已有的预览脚本也可以完成类似工作：

代码块 powershell

```
1 .\.venv\Scripts\python.exe scripts\ingest.py --dry-run --sample-size 1
```

其中 dry-run 表示先预览处理结果，不写入向量数据库。

4.5 离线阶段与在线阶段

离线阶段不是一定断开网络，而是指提前准备知识库，不在每次用户查询时重复完成全部准备工作。

本项目的资料准备过程可以这样理解：

代码块 text

1 CSV → Loader → Document → Embedding / BM25 准备 → Milvus 索引

在线阶段指收到用户问题后，使用已经准备好的知识库进行查询：

代码块 text

1 用户问题 → 查询理解与改写 → 召回 → 精排 → 澄清 / 结果展示

第一次加载本地模型、首次下载权重和首次建索引可能耗时较长。这些准备工作和正常查询耗时要分开看。

4.6 先认识检索链路中的名词

这一节先知道每个组件做什么，后续章节再学习公式、参数和效果对比。

名词	中文解释	本项目的作用
Dense Retrieval	稠密向量检索	通过 Embedding 表达文本语义，召回相关资料
BM25	基于词项统计的检索评分方法	通过关键词、型号等词项召回候选
Hybrid Retrieval	混合检索	同时使用语义与词项检索结果
RRF	Reciprocal Rank Fusion，倒数排名融合	根据各路结果的名次融合候选列表
Reranker	重排序模型	对召回候选重新判断相关性并排序
Metadata Filter	元数据过滤	根据提取的条件缩小查询范围
Milvus	向量数据库	保存索引并执行检索，不生成对话回答

为什么不只用一种检索？车辆资料既包含口语表达，也包含 EDC7、4HK1 等型号编码。语义匹配和词项匹配各有作用，组合后是否更好还需要评估验证。

注意

Metadata Filter 可能过严；Reranker 也可能排错。增加组件不意味着一定更准确。当前代码在带过滤的精排结果为空时会尝试无过滤回退，详细逻辑留到检索章节学习。

4.7 本节回顾

请回答：为什么 CSV 的一行可以成为一个 Document？为什么本项目不需要默认切分电路图 PDF？Milvus、Embedding 和 Reranker 分别负责什么？

5. Agent：让模型使用工具

5.1 什么是 Agent

Agent，智能体，在本章可以理解为：模型根据任务选择工具，程序执行工具，再把结果交回模型处理的系统。

模型不只是生成文字，还可以输出“我要调用哪个工具、参数是什么”。框架负责执行这个请求，并把结果组织成工具消息。

LangChain 的 `create_agent()` 提供这种模型与工具循环的构建方式，参数包括 `model`、`tools` 和 `system_prompt`。参见 [LangChain Agents 文档](#)。

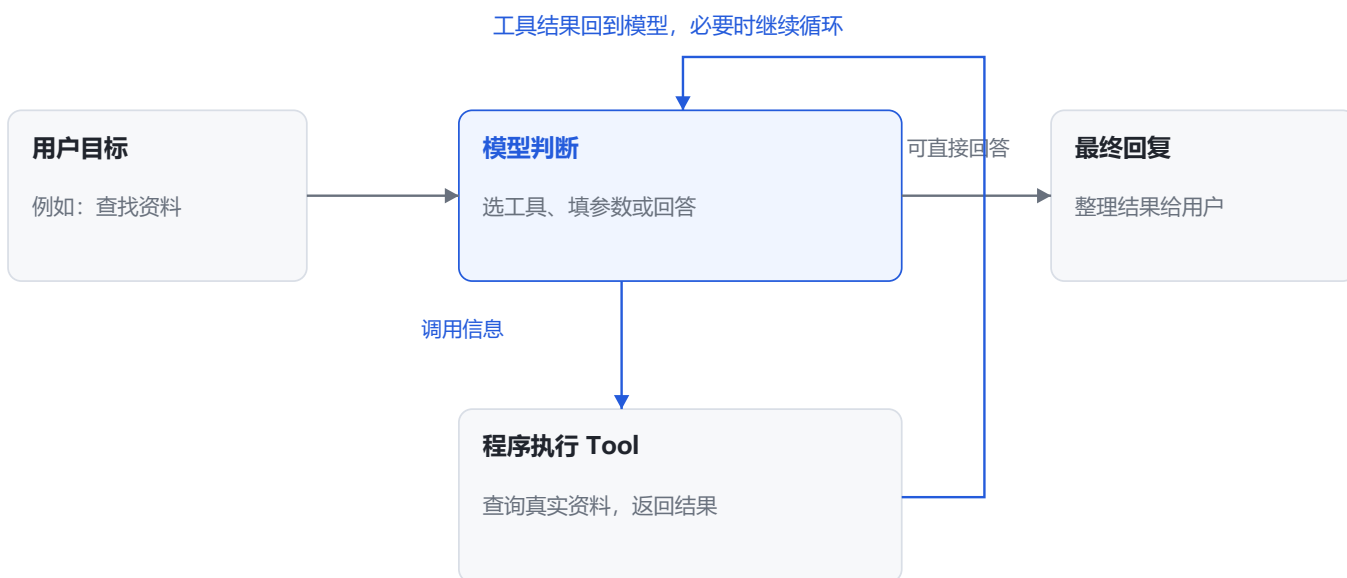


图1-4 Agent 工具循环：模型提出调用，程序执行工具

应用场景： 同一助手可能需要搜索资料、按 ID 查询记录，也可能直接回答普通问候，希望由模型选择合适的操作。

项目对应： `app/agents/circuit_agent.py`。

5.2 Tool 是什么

Tool，工具，是提供给 Agent 调用的可执行能力。模型需要知道工具名称、用途和参数格式，才能提出调用请求。

在课程中，我们常用 `@tool` 装饰器描述工具。项目也采用这个写法。

下面是一个独立的教学示例，只验证工具机制，不执行资料检索：

代码块 python

```
1 from langchain.tools import tool
2
3
4 @tool
5 def normalize_query(query: str) -> str:
6     """去掉查询首尾的空白，保留车型与型号的原始写法。"""
7     return query.strip()
8
9
10 result = normalize_query.invoke({"query": " EDC7 电路图 "})
11 print(result)
```

运行后应得到：

代码块 text

```
1 EDC7 电路图
```

这里有三个需要观察的细节：

- `normalize_query` 是工具名称，使用 `snake_case` 命名。
- `query: str` 表达参数类型，帮助形成工具输入结构。
- 三引号说明是工具用途的描述，不只是给程序员看的注释。

被 `@tool` 装饰后，我们在这个示例中用 `.invoke({...})` 执行工具，而不是把它当作原始普通函数直接调用。

5.3 项目里有哪些工具

当前独立 Agent 绑定了两个工具：

工具名	主要参数	做什么
search_knowledge_base	query、top_k	理解与改写查询，执行资料检索，返回资料信息
get_document_by_id	document_id	从本地资料索引按 ID 获取标题和层级路径

源码阅读： `app/tools/search_knowledge_base.py`、`app/tools/get_document.py`。

这里的按 ID 工具查询的是现有资料记录，不调用对话模型，也不需要执行向量搜索。这很适合用作第一个工具实验。

5.4 代码示例：执行真实工具

运行条件： 项目依赖和 CSV 已准备好。这个实验不需要 DeepSeek 和 Milvus。

练习文件名（自行新建）： `ch01_tool_call.py`。

代码块 python

```
1 import json
2
3 from app.rag.loader import load_circuit_documents
4 from app.tools.get_document import get_document_by_id
5
6
7 def main() -> None:
8     # 1. 从真实数据中取一个存在的文档 ID
9     docs = load_circuit_documents().documents
10    if not docs:
11        raise RuntimeError("CSV 没有可用的资料记录")
12    document_id = int(docs[0].metadata["doc_id"])
13
14    # 2. 查看工具名称、描述和参数名
15    print("工具名称: ", get_document_by_id.name)
16    print("工具描述: ", get_document_by_id.description)
17    print("参数名称: ", list(get_document_by_id.args))
18
19    # 3. 直接执行工具，不调用对话模型
20    payload = get_document_by_id.invoke({"document_id": document_id})
21    print(json.dumps(json.loads(payload), ensure_ascii=False, indent=2))
22
23
24 if __name__ == "__main__":
25     main()
```

运行命令：

代码块 powershell

```
1 .\venv\Scripts\python.exe ch01_tool_call.py
```

结果解释： 应看到工具名称、描述和 document_id 参数；随后返回 status="found" 和真实资料元数据。
json.loads() 把工具返回的 JSON 字符串转成 Python 对象，随后格式化打印。

这个实验说明“工具能执行”，但还没有展示“模型自主选择工具”。两者不要混淆。

5.5 create_agent 的基本结构

项目已经封装好 get_circuit_agent()。下面是 app/agents/circuit_agent.py 中的创建片段，model 通过 get_chat_model() 获得：

代码块 python

```
1 # 1. 准备模型可以调用的工具
2 tools = [
3     search_knowledge_base,
4     get_document_by_id,
5 ]
6
7 # 2. 创建 Agent
8 agent = create_agent(
9     model=model,
10    tools=tools,
11    system_prompt=SYSTEM_PROMPT,
12 )
```

你可以把它理解为三项输入：

- **Model:** 使用哪个模型判断任务。
- **Tools:** 提供哪些可以执行的能力。
- **System Prompt:** 给模型设定任务和工具使用要求。

工具执行后的结果还会交回模型，因此 Agent 返回的最后一消息通常是模型整理后的回复，不是工具原始 JSON。

5.6 可选实验：运行独立 Agent

完整 Agent 实验放在本节最后，不作为读懂本章的前提。

运行条件： .env 配置有效的模型凭证；如果问题触发检索，Milvus、索引、Embedding 和 Reranker 也必须可用。该实验会调用模型服务。

代码块 powershell

```
1 # 1. 先在项目根目录加载已有环境变量
2 . .\scripts\import-dotenv.ps1 -Path .\.env
3
4 # 2. 运行仓库已有的独立 Agent 脚本
5 .\.venv\Scripts\python.exe scripts\run_agent.py "东风天龙仪表针脚图"
```

这条命令运行的是独立 Agent，不是 /api/chat 的 Web 主 workflow。本章没有宣称该实验已在你的模型服务和数据库环境中重新执行成功。

5.7 Agent 与 RAG 是什么关系

Agent 解决的是“下一步使用什么能力”；RAG 解决的是“如何利用外部知识作为回答依据”。

它们可以组合：Agent 将知识库检索作为一个工具，需要资料时调用，再根据工具结果回复。这通常称为 Agentic RAG。

但 RAG 不要求一定使用 Agent。一个由程序固定执行检索与生成的流程，同样可以采用 RAG。也不是每个 Agent 都必须拥有向量数据库。

5.8 本节回顾

你应该能说明：模型提出工具调用与程序执行工具的区别，@tool 的作用，以及“直接执行工具”和“运行 Agent”为什么是两个实验。

6. LangGraph：把检索过程组织成 workflow

6.1 为什么还需要 workflow

假设系统已经找到 20 条资料。我们希望先询问用户要电路图还是引脚定义，而不是随意展示 20 条结果。

这类规则不适合只在 Prompt 里说一句“结果太多就问用户”。我们需要程序判断数量、生成真实选项、保存当前候选，并在用户选择后继续处理。

Workflow，工作流，就是组织多个处理步骤及其分支的执行流程。

LangGraph 是项目用于描述和执行这类有状态流程的框架。它可以包含模型调用，也可以包含数据库访问、普通计算和人工输入。

6.2 先认识三个核心名词

名词	含义	项目例子
State	节点之间共享的流程数据	原始问题、候选资料、当前选项
Node	一个处理步骤	understand_query、retrieve、clarify
Edge	节点之间的执行连接	检索后按数量进入不同节点

项目对应： app/graph/state.py、app/graph/workflow.py、app/graph/nodes/。

第一章先认识职责，不需要背下完整 StateGraph API。

6.3 当前 Web 主流程

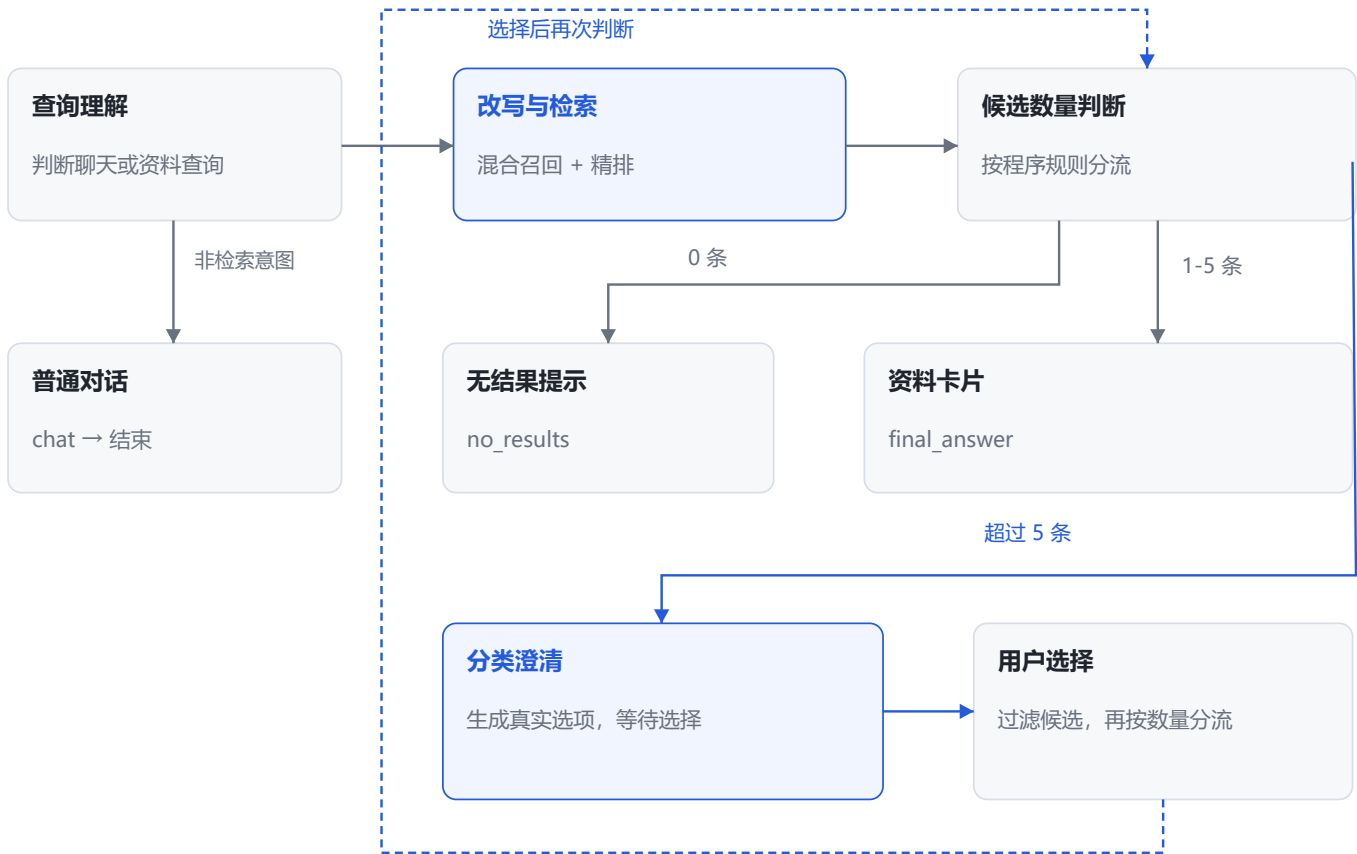


图1-5 Web 主流程：按候选数量分流，用户选择后继续收敛

build_facets 负责生成分类选项，clarify 负责等待选择并处理候选收敛。用户选择后，流程还会根据数量决定下一步。

Facet，分类维度，是用于分组候选的属性，例如资料类型或层级分类。它不是让模型凭空编造出来的菜单。

6.4 一个小实验：验证业务分支

我们不运行整个 Graph，只调用项目里按候选数量分流的真实函数。

练习文件名（自行新建）： ch01_route_demo.py。

代码块 python

```
1 from app.graph.nodes.retrieve import route_by_result_count
2
3
4 def main() -> None:
5     # 1. 用合成候选数量验证路由, 不执行真实检索
6     for count in (0, 3, 8):
7         state = {"candidate_documents": [{_ for _ in range(count)]}
8
9         # 2. 调用项目中的真实路由函数
10        next_node = route_by_result_count(state)
11        print(f"候选数量 {count} -> {next_node}")
12
13
14 if __name__ == "__main__":
15     main()
```

运行命令:

代码块 powershell

```
1 .\venv\Scripts\python.exe ch01_route_demo.py
```

默认 GRAPH_MAX_RESULTS=5 时, 预期结果:

代码块 text

```
1 候选数量 0 -> no_results
2 候选数量 3 -> final_answer
3 候选数量 8 -> build_facets
```

这里的空字典只用于构造候选数量, 不是真实资料, 也没有验证检索质量。如果你改过展示上限, 3 条和 8 条的分支可能随配置改变。

6.5 interrupt 与 resume 先怎么理解

Interrupt, 中断, 表示流程暂停在一个需要外部输入的位置。

Resume, 恢复, 表示把用户的选择交回流程, 让它继续执行。

Checkpoint, 检查点保存器, 用来保存流程状态。`thread_id` 用来区分会话, 使恢复能找到对应的状态。

代码块 text

```
1 检索候选 → 生成选项 → interrupt() 暂停
2                                     ↓
3                             用户选择
4                                     ↓
5                     Command(resume=选项值) → 继续处理
```

在当前 Web 中, 前端选择通过 `/api/select` 进入 `resume_search_workflow()`。选项值必须来自当前流程提供的真实选项, 不是随意输入一个数字。

注意

Web 主流程使用 `InMemorySaver`，状态保存在进程内存中。它不等于跨进程重启的持久化存储，也不自动等于长期用户记忆。Studio 使用单独导出的图，由其运行环境提供检查点管理。

6.6 两个入口不要混淆

入口	使用方式	谁控制主要流程
Web: <code>/api/chat</code>	<code>run_search_workflow()</code>	自定义 <code>LangGraph</code> 节点与条件分支
独立 Agent: <code>scripts/run_agent.py</code>	<code>run_circuit_agent()</code>	<code>create_agent()</code> 的模型与工具循环

这不是谁替代谁的问题。项目保留了工具式 Agent 能力，但当前 Web 为了明确实现多轮澄清规则，使用自定义工作流。

第一章最重要的阅读习惯是：**从实际入口追调用链，不仅根据仓库里有没有某个函数判断系统在用什么。**

6.7 本节回顾

你应该能说明 State、Node、Edge 的含义，解释“超过 5 条继续澄清”的规则放在哪里，并区分 Web 和独立 Agent 的执行入口。

7. 命名规范与代码阅读顺序

7.1 先统一变量和函数命名

课程代码常用简短的业务词汇，例如 `model`、`messages`、`query`、`docs`、`agent`。项目中仍然可以使用这些名字，再用模块职责区分上下文。

推荐在新增 Python 学习代码中使用 `snake_case`：单词小写，用下划线连接。

类型	规则	项目例子
文件、函数、变量	<code>snake_case</code>	<code>query_pipeline.py</code> 、 <code>get_chat_model</code> 、 <code>original_query</code>
类与数据结构	<code>PascalCase</code>	<code>Settings</code> 、 <code>SearchIntent</code> 、 <code>CircuitSearchState</code>
常量与环境变量	<code>UPPER_SNAKE_CASE</code>	<code>SYSTEM_PROMPT</code> 、 <code>DEEPSEEK_API_KEY</code>
图节点名称	按动作与职责命名	<code>understand_query</code> 、 <code>retrieve</code> 、 <code>clarify</code>

例如，不要把学习代码写成含义不清的 `data1`、`data2`、`doSomething()`：

代码块 python

```
1 # 问题本身
2 query = "东风天龙仪表盘针脚图"
3
4 # 模型处理后的检索问题
5 rewritten_query = query.strip()
6
7 # 候选资料与最终资料使用不同名字
8 candidate_documents = []
9 final_results = []
```

代码中 `rewritten_query = query.strip()` 只是命名示意；项目真实 Query Rewrite 由查询处理模块完成，不是只去掉空格。

7.2 用名称表达动作和结果

几个常用前缀可以帮助你阅读代码：

- `get_`：获取已配置的对象或记录，例如 `get_chat_model()`。
- `load_`：从文件或数据源加载内容，例如 `load_circuit_documents()`。
- `parse_`：把输入解析成结构，例如 `parse_search_intent()`。
- `build_`：构建对象或选项，例如 `build_circuit_graph()`。
- `run_`：运行一段完整流程，例如 `run_search_workflow()`。
- `resume_`：恢复暂停流程，例如 `resume_search_workflow()`。

这些是命名习惯，不是 Python 强制语法。一个好名字应当让你猜到函数的职责，再通过实际代码确认。

7.3 注释要解释步骤和意图

飞书课程常用 # 1.、# 2. 拆解实验。本章也保留这种风格，方便你逐步阅读。

代码块 python

```
1 # 1. 读取资料，先确认数据结构
2 result = load_circuit_documents()
3
4 # 2. 取一条样本，避免一次打印整个资料库
5 doc = result.documents[0]
6
7 # 3. 分开观察检索文本与元数据
8 print(doc.page_content)
9 print(doc.metadata)
```

注释应该告诉你这一步为什么存在。仅把 `query.strip()` 注释为“调用 strip 方法”，并不能帮助理解业务。

7.4 现有前端的 camelCase 怎么办

项目保留了一些前端和 API 字段，例如 `sessionId`、`fileName`、`hierarchyPath`。它们是现有接口约定，不应该为了统一 Python 命名随意修改。

内部数据使用 `doc_id`、`hierarchy_path`；对外展示时通过转换函数适配前端格式。

项目对应： app/graph/service.py 中的 _frontend_document()。

学习时先认清“内部变量”和“对外字段”两个层次。命名规范应帮助理解和维护代码，而不是破坏已有接口。

7.5 先读哪些文件

第一轮不需要逐行读整个仓库，按下面的顺序找到业务主线即可：

顺序	代码位置	带着这个问题阅读
1	app/api/chat.py	问题和用户选择从哪里进入？
2	app/graph/service.py	怎样启动、恢复流程，并组装响应？
3	app/graph/workflow.py	节点怎样连接，分支在哪里？
4	app/core/model.py	模型配置如何变成可调用对象？
5	app/rag/loader.py	一条 CSV 记录怎样成为 Document？
6	app/agents/circuit_agent.py	独立 Agent 绑定了哪些工具？

随后再分专题读 app/rag/ 和 app/graph/nodes/。检索策略、状态恢复等细节有了位置感后，再看实现会容易得多。

7.6 本节回顾

你应该能辨认 query、rewritten_query、candidate_documents 和 final_results，并解释为什么已有 sessionId 字段不应随意改成另一个名字。

8. 动手练习与本章回顾

8.1 准备学习环境

本项目声明 Python 版本范围为 >=3.11,<3.13。学习时使用已有 .venv，无需照课程中其他演示项目的版本重新创建环境。

所有本章命令都从项目根目录运行：

代码块 powershell

```
1 Set-Location -LiteralPath 'D:\Study\chatbot-circuit-diagram'
2
3 # 查看当前解释器版本
4 .\.venv\Scripts\python.exe --version
```

如果 .venv 或依赖尚未准备好，再按项目 README 的首次安装步骤处理。本章不要求改用 uv，也不要求新装 Notebook；Python 文件同样可以完成实验。

实验	模型 API	Milvus / 索引	本地 CSV
Document 预览	不需要	不需要	需要
按 ID 执行 Tool	不需要	不需要	需要
候选数量路由	不需要	不需要	不需要
单次模型调用	需要	不需要	不需要
独立 Agent 检索	需要	需要	需要
Web 完整流程	需要	需要	需要

路由实验需要已安装项目依赖，但不会加载模型权重或执行真实检索。本地导入可能需要少量时间。

8.2 练习一：从真实数据认识 Document

执行 `ch01_document_preview`，然后完成三个任务：

- 1. 找到 `page_content` 中的标题和分类路径。
- 2. 找到 `metadata` 中同一条资料的 ID。
- 3. 解释为什么这两个字段可以同时存在，不能互相替代。

完成标准： 你可以拿着一条打印结果，向别人解释这条资料如何参与检索、如何回到原记录。

8.3 练习二：区分工具与模型

执行 `ch01_tool_call`，观察工具名称、描述、参数和结果。

然后回答：工具直接返回的 JSON 和 Agent 最终生成的自然语言回复，有什么不同？为什么这个实验不需要模型 API？

完成标准： 能独立说明“查到真实资料”来自程序执行工具，“怎样整理回复”可以来自模型。

8.4 练习三：验证分支规则

执行 `ch01_route_demo`。打开 `route_by_result_count()`，找出决定返回哪个节点的条件。

不用改配置文件，先把学习脚本里的数量改成 1、5、6，再观察默认上限下的分支。

完成标准： 能说明这段实验只验证路由规则，没有证明检索准确。

8.5 练习四：观察模型调用

环境准备好后，运行 `ch01_model_call`，观察消息类型和正文。然后在学习脚本中修改系统消息，比较模型的表达变化。

不要把“模型回答更像助手”当作“已经会查资料”。本节实验没有调用资料检索工具。

完成标准： 能区分模型的角色约束、自然语言回复和资料库事实。

8.6 可选：在 Web 端寻找本章概念

如果完整运行环境已经准备好，可以使用项目现有脚本：

```
代码块 powershell
1 .\scripts\start.ps1
```

然后打开默认地址 <http://127.0.0.1:8011>，尝试普通问候和资料查询。

启动脚本负责启动 Docker 数据服务及 Web；它不会自动完成新环境的首次索引构建。数据库和索引尚未就绪时，应先参考 README。

观察普通问候、无结果提示、资料卡片、澄清选项分别属于哪种响应。具体结果由实际数据和模型执行决定，不要要求每个查询都进入澄清。

8.7 常见问题

问题：为什么我执行学习脚本时提示找不到模块？

先确认已经在项目根目录新建练习文件、复制完整代码，然后从同一目录执行文档给出的命令。建议使用项目的 `.venv` 解释器；把练习文件放在其他目录直接运行，可能造成项目包不在导入路径中。

问题：为什么模型调用提示没有配置 Key？

确认根目录 `.env` 已配置有效凭证。`Settings` 会读取 `.env`，但已经创建的配置对象和模型有缓存，修改后需要重启实验进程。

问题：为什么资料数量和别人的截图不同？

预览结果来自你当前 CSV；数据文件可能不同。先检查自己的输入数据，不要为了对齐截图修改真实结果。

问题：为什么代码里有 Agent，Web 却不直接调用它？

仓库保留了独立 Agent；当前 Web 主链路调用的是自定义 `LangGraph`，以明确执行多轮选择和结果展示规则。两者应分别从入口理解。

8.8 自测题

1. 大语言模型、Embedding 和 Reranker 的输出分别是什么？
2. 给模型写“不要编造资料”，是否已经能保证资料真实？
3. RAG 是一种模型、一个框架，还是一种应用架构思路？
4. 一条 CSV 记录为什么可以成为一个 Document？
5. 模型是否亲自执行 Python 工具？
6. 当前 Web 入口实际使用 `create_agent()` 还是自定义 workflow？
7. 5 条展示上限和 30 条候选上限有什么不同？
8. `InMemorySaver` 能否保证服务重启后继续恢复旧会话？

8.9 参考答案

1. 对话模型输出消息或结构化信息；Embedding 输出数值向量；Reranker 输出相关性评分或排序结果。

2. 不能。还需要从真实数据获取结果，并由程序校验和组织输出。
3. 是应用架构思路，通常通过检索外部知识增强生成。
4. 它已经包含一条独立资料的文本和元数据，不要求是完整 PDF。
5. 不亲自执行。模型提出调用信息，框架或程序执行函数并返回结果。
6. 当前 Web 使用自定义 LangGraph，独立 Agent 入口另行保留。
7. 候选用于排序与澄清，展示上限限制最终给用户看的资料条数；这些是默认配置。
8. 不能。当前 Web 的状态保存在进程内存中，重启后的持久化需要其他保存机制。

8.10 下一章怎么学

下一章建议学习 **“资料进入检索系统：Loader、Embedding 与 Milvus”**。沿着一条真实资料，从 CSV 到 Document、再到索引，理解 Dense 与 BM25 的作用。

随后学习混合召回、RRF、Reranker 和 Query Rewrite；再深入 Agent 工具循环与 LangGraph 多轮恢复。这个顺序围绕项目依赖安排，不强行按课程章节逐条复刻。

附录. 本章来源与阅读定位

A. 飞书资料中对应的主题

本章参考原资料的讲解结构和命名方式，以项目场景重新组织内容，没有复制无关的历史介绍、产品列表或课程品牌图片。

- 《第1章. AI通识与基础》：大模型应用、API 调用、消息角色、无状态对话与环境变量。
- 《第1节. LangChain核心组件》：Agent、模型、Messages、Prompt、Tools 和基础记忆。
- 《第2节. Agent入门实战（AI私厨）》：需求分析、实现思路、先实验后工程化的写法。
- 《第1节 RAG Agent》：外部知识、Document、离线与在线流程。
- 《第2节 RAG查询优化》：固定检索流程与 Agentic RAG 的区分。
- 《第4节 Milvus向量库》：向量库、稠密检索、BM25、融合和精排的职责。
- 《第0节 LangGraph》《第1节. 基本概念和入门》：工作流、State、Node、Edge 与人工中断。
- 《LangChain_Agent_RAG_LangGraph_项目改造全量参考手册》：模块化、命名、注释和项目边界。

原资料中涉及模型能力、版本、服务价格等内容可能具有时效性。本章只保留理解本项目需要的概念与用法，接口以实际项目及官方说明为准。

B. 官方补充阅读

- [LangChain Agents](#)：模型与工具循环、`create_agent()`。
- [LangChain create_agent API](#)：参数与调用接口。
- [DeepSeek Multi-round Conversation](#)：无状态 API 与消息历史。

C. 项目源码索引

- `app/core/config.py`、`app/core/model.py`：配置读取与对话模型。
- `app/rag/embeddings.py`、`app/rag/reranker.py`：向量与精排模型。
- `app/rag/loader.py`：CSV 转 Document。
- `app/tools/get_document.py`、`app/tools/search_knowledge_base.py`：资料工具。
- `app/agents/circuit_agent.py`：独立 Agent。
- `app/api/chat.py`、`app/graph/service.py`：Web 入口与流程调用。
- `app/graph/state.py`、`app/graph/workflow.py`：状态与工作流结构。
- `app/graph/nodes/retrieve.py`、`app/graph/nodes/answer.py`：检索、分支与资料输出。

本章代码保留在 PDF 内，不附独立脚本。运行时在项目根目录自行新建练习文件即可，不需要改动业务源码；网络实验由你在环境准备好后执行。

D. 本章实验校验

2026-10-03 本地离线实验已验证：4233 条有效资料、0 条无效记录；按 ID 工具返回真实记录；0、3、8 条候选的分支符合默认规则。

模型示例通过语法检查；本次未重新调用模型 API、独立 Agent 或 Web 全链路。